

1. Write a C program to simulate the following preemptive CPU Scheduling algorithms to find turnaround time, waiting time, average turnaround time and average waiting time.

a) FCFS b) SRTF c) Priority Scheduling (Preemptive and Non preemptive) d) SJF

Take required inputs from user such as process id, burst time, arrival time.

```
import copy
class Process:
    def __init__(self, pid, at, bt, priority):
        self.pid = pid
        self.at = at
        self.bt = bt
        self.remaining = bt
        self.priority = priority
        self.ct = 0
        self.tat = 0
        self.wt = 0
def calculate(p):
    p.tat = p.ct - p.at
    p.wt = p.tat - p.bt
def print_results(processes):
    total_wt = total_tat = 0
    print("\nPID\tAT\tBT\tCT\tTAT\tWT")
    for p in processes:
        print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")
        total_wt += p.wt
        total_tat += p.tat
    print("\nAverage Waiting Time:", round(total_wt/len(processes),2))
    print("Average Turnaround Time:", round(total_tat/len(processes),2))
def input_processes():
    n = int(input("Enter number of processes: "))
    processes = []
    for i in range(n):
        pid = input("Process ID: ")
        at = int(input("Arrival Time: "))
        bt = int(input("Burst Time: "))
        pr = int(input("Priority: "))
        processes.append(Process(pid, at, bt, pr))
    return processes
# ----- FCFS ----- #
def fcfs(processes):
    processes.sort(key=lambda x: x.at)
    time = 0
    for p in processes:
        if time < p.at:
            time = p.at
        time += p.bt
        p.ct = time
        calculate(p)
    print("\n==== FCFS =====")
    print_results(processes)
# ----- SJF ----- #
def sjf(processes):
    time = 0
    completed = 0
    n = len(processes)
    while completed < n:
        ready = [p for p in processes if p.at <= time and p.ct == 0]
        if not ready:
            time += 1
            continue
        p = min(ready, key=lambda x: x.bt)
        time += p.bt
        p.ct = time
```

```

        calculate(p)
        completed += 1
    print("\n===== SJF =====")
    print_results(processes)
# ----- SRTF ----- #
def srtf(processes):
    time = 0
    completed = 0
    n = len(processes)
    while completed < n:
        ready = [p for p in processes if p.at <= time and p.remaining >
0]

        if not ready:
            time += 1
            continue
        p = min(ready, key=lambda x: x.remaining)
        p.remaining -= 1
        time += 1
        if p.remaining == 0:
            p.ct = time
            calculate(p)
            completed += 1
    print("\n===== SRTF =====")
    print_results(processes)
# ----- Priority Non-Preemptive ----- #
def priority_np(processes):
    time = 0
    completed = 0
    n = len(processes)
    while completed < n:
        ready = [p for p in processes if p.at <= time and p.ct == 0]
        if not ready:
            time += 1
            continue
        p = min(ready, key=lambda x: x.priority)
        time += p.bt
        p.ct = time
        calculate(p)
        completed += 1
    print("\n===== Priority Non-Preemptive =====")
    print_results(processes)
# ----- Priority Preemptive ----- #
def priority_p(processes):
    time = 0
    completed = 0
    n = len(processes)
    while completed < n:
        ready = [p for p in processes if p.at <= time and p.remaining >
0]

        if not ready:
            time += 1
            continue
        p = min(ready, key=lambda x: x.priority)
        p.remaining -= 1
        time += 1
        if p.remaining == 0:
            p.ct = time
            calculate(p)
            completed += 1
    print("\n===== Priority Preemptive =====")
    print_results(processes)
# ----- MAIN ----- #
def main():
    processes = input_processes()
    print("\nChoose Algorithm:")
    print("1.FCFS 2.SJF 3.SRTF 4.Priority NP 5.Priority P")
    ch = int(input("Enter choice: "))
    pcopy = copy.deepcopy(processes)
    if ch == 1:
        fcfs(pcopy)
    elif ch == 2:
        sjf(pcopy)
    elif ch == 3:
        srtf(pcopy)
    elif ch == 4:

```

```

        priority_np(pcopy)
    elif ch == 5:
        priority_p(pcopy)
if __name__ == "__main__":
    main()

```

```

● PS C:\Users\hp\OS_LAB> & c:\python314\python.exe "c:/Users/hp/OS_LAB/lab3&4/OS_5.1.py"
Enter number of processes: 3
Process ID: 1
Arrival Time: 2
Burst Time: 5
Priority: 2
Process ID: 2
Arrival Time: 5
Burst Time: 6
Priority: 1
Process ID: 3
Arrival Time: 2
Burst Time: 3
Priority: 3

Choose Algorithm:
1.FCFS 2.SJF 3.SRTF 4.Priority NP 5.Priority P
Enter choice: 4

===== Priority Non-Preemptive =====

PID    AT    BT    CT    TAT    WT
1       2     5     7     5     0
2       5     6    13     8     2
3       2     3    16    14    11

Average Waiting Time: 4.33
Average Turnaround Time: 9.0
○ PS C:\Users\hp\OS_LAB>

```

2. Implementation of CPU Scheduling (Pre-emptive Processes): i) Round Robin ii) Longest job first and iii) Longest Remaining Time First

```

import copy
class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid
        self.at = at
        self.bt = bt
        self.remaining = bt
        self.ct = 0
        self.tat = 0
        self.wt = 0
def calculate(p):
    p.tat = p.ct - p.at
    p.wt = p.tat - p.bt
def print_results(processes):
    total_wt = total_tat = 0
    print("\nPID\tAT\tBT\tCT\tTAT\tWT")
    for p in processes:
        print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")
        total_wt += p.wt
        total_tat += p.tat
    print("\nAverage Waiting Time:", round(total_wt/len(processes),2))

```

```

    print("Average Turnaround Time:", round(total_tat/len(processes),2))
def input_processes():
    n = int(input("Enter number of processes: "))
    processes = []
    for i in range(n):
        pid = input("Process ID: ")
        at = int(input("Arrival Time: "))
        bt = int(input("Burst Time: "))
        processes.append(Process(pid, at, bt))
    return processes
# ----- Round Robin ----- #
def round_robin(processes, quantum):
    time = 0
    queue = []
    processes.sort(key=lambda x: x.at)
    queue.append(processes[0])
    i = 1
    while queue:
        p = queue.pop(0)
        run = min(quantum, p.remaining)
        time += run
        p.remaining -= run
        while i < len(processes) and processes[i].at <= time:
            queue.append(processes[i])
            i += 1
        if p.remaining > 0:
            queue.append(p)
        else:
            p.ct = time
            calculate(p)
    print("\n===== Round Robin =====")
    print_results(processes)
# ----- Longest Job First ----- #
def ljf(processes):
    processes.sort(key=lambda x: -x.bt)
    time = 0
    for p in processes:
        if time < p.at:
            time = p.at
        time += p.bt
        p.ct = time
        calculate(p)
    print("\n===== Longest Job First =====")
    print_results(processes)
# ----- Longest Remaining Time First ----- #
def lrtf(processes):
    time = 0
    completed = 0
    n = len(processes)
    while completed < n:
        ready = [p for p in processes if p.at <= time and p.remaining >
0]

        if not ready:
            time += 1
            continue
        p = max(ready, key=lambda x: x.remaining)
        p.remaining -= 1
        time += 1
        if p.remaining == 0:
            p.ct = time
            calculate(p)
            completed += 1
    print("\n===== Longest Remaining Time First =====")
    print_results(processes)
# ----- MAIN ----- #
def main():
    processes = input_processes()
    print("\n1.Round Robin 2.LJF 3.LRTF")
    ch = int(input("Enter choice: "))
    pcopy = copy.deepcopy(processes)
    if ch == 1:
        q = int(input("Enter Quantum: "))
        round_robin(pcopy, q)
    elif ch == 2:
        ljf(pcopy)

```

```

elif ch == 3:
    lrtf(pcopy)
if __name__ == "__main__":
    main()

```

```

● PS C:\Users\hp\OS_LAB> & c:\python314\python.exe "c:/Users/hp/OS_LAB/lab3&4/OS_5.2.py"
Enter number of processes: 3
Process ID: 1
Arrival Time: 2
Burst Time: 3
Process ID: 2
Arrival Time: 3
Burst Time: 1
Process ID: 3
Arrival Time: 2
Burst Time: 1

1.Round Robin 2.LJF 3.LRTF
Enter choice: 1
Enter Quantum: 1

===== Round Robin =====

PID    AT    BT    CT    TAT    WT
1       2     3     4     2     -1
3       2     1     3     1     0
2       3     1     5     2     1

Average Waiting Time: 0.0
Average Turnaround Time: 1.67
○ PS C:\Users\hp\OS_LAB>

```